

# Syntax Analysis

## 1. Introduction to Syntax Analysis

### 1.1 What is Syntax Analysis?

**Definition:** Syntax Analysis (also called **Parsing**) is the second phase of compilation that checks whether the sequence of tokens follows the grammatical rules of the programming language.

**Simple Explanation:** After the lexical analyzer breaks code into tokens, the syntax analyzer checks if those tokens are arranged in a grammatically correct order.

**Alternative Name:** The Syntax Analyzer is also called a **Parser**.

### 1.2 From Lexical to Syntax Analysis

**Example Flow:**

SOURCE CODE:

```
x = a + b * 3;
```

↓ [LEXICAL ANALYSIS]

TOKEN STREAM:

```
ID(x) = ID(a) + ID(b) * NUM(3) ;
```

Token 1: IDENTIFIER with value "x"

Token 2: OPERATOR "="

Token 3: IDENTIFIER with value "a"

Token 4: OPERATOR "+"

Token 5: IDENTIFIER with value "b"

Token 6: OPERATOR "\*"

Token 7: NUMBER with value "3"

Token 8: DELIMITER ";"

↓ [SYNTAX ANALYSIS]

PARSE TREE:

Checks if tokens follow grammar rules

Builds tree structure showing program organization

## 2. Why Do We Need Syntax Analysis?

### 2.1 Purpose of Syntax Analysis

Programming languages have **structure** - not just any sequence of valid tokens is acceptable.

**Analogy:**

- **Lexical Analysis** = Spell checking (are individual words correct?)
- **Syntax Analysis** = Grammar checking (are sentences structured correctly?)

### 2.2 Valid vs Invalid Structures

#### Example 1: If Statement

VALID:

if condition then statement

INVALID:

if then condition statement

**Reason:** Keywords must appear in correct order

#### Example 2: Expression

VALID:

$a + b * c$

INVALID:

$a + * b c$

$+ a b * c$

**Reason:** Operators and operands must follow rules

#### Example 3: Assignment

VALID:

$x = y + 5;$

INVALID:

$= x y 5 +;$

$x y = + 5;$

**Reason:** Assignment operator and expression structure matters

## 2.3 Summary

LEXICAL ANALYZER checks:

- Are individual tokens valid?
- Is "while" spelled correctly?
- Is "123" a valid number?

SYNTAX ANALYZER checks:

- Are tokens arranged correctly?
- Does "while (condition) { body }" follow grammar rules?
- Is the expression structure valid?

## 3. What is a Grammar?

### 3.1 Definition

**Grammar:** A set of rules that defines the valid sentences (programs) in a language.

**Formal Definition:** A grammar is used to specify the syntax of a language using productions (rules). **Purpose:** Defines what constitutes a syntactically correct program.

### 3.2 Grammar Components

A Context-Free Grammar (CFG) consists of:

#### 1. TERMINALS (T)

- Tokens from lexical analysis
- Examples: id, num, +, \*, (, )

#### 2. NON-TERMINALS (V)

- Variables representing language constructs
- Examples: E (expression), T (term), F (factor)

#### 3. PRODUCTIONS (P)

- Rules showing how to form valid structures
- Format: Non-terminal → string of terminals/non-terminals

#### 4. START SYMBOL (S)

- The top-level non-terminal

- Example: E (expression)

## 4. Example Grammar

### 4.1 Expression Grammar

#### Grammar Definition:

$E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$

$F \rightarrow ( E ) \mid \text{id}$

Where:

- E = Expression
- T = Term
- F = Factor
- id = identifier (variable name)

### 4.2 What This Grammar Says

#### Rule 1: $E \rightarrow E + T \mid T$

An Expression can be:

- Another Expression plus a Term ( $E + T$ )
- OR just a Term ( $T$ )

Examples:

- "id" is an Expression (through  $E \rightarrow T \rightarrow F \rightarrow \text{id}$ )
- "id + id" is an Expression ( $E + T$ )
- "id + id + id" is an Expression ( $E + T$ , then  $E + T$  again)

#### Rule 2: $T \rightarrow T * F \mid F$

A Term can be:

- Another Term times a Factor ( $T * F$ )
- OR just a Factor ( $F$ )

Examples:

- "id" is a Term (through  $T \rightarrow F \rightarrow \text{id}$ )
- "id \* id" is a Term ( $T * F$ )

- "id \* id \* id" is a Term ( $T \rightarrow F$ , then  $T \rightarrow F$  again)

### **Rule 3: $F \rightarrow ( E ) \mid id$**

A Factor can be:

- A parenthesized Expression (  $E$  )
- OR an identifier (id)

Examples:

- "id" is a Factor
- "(id)" is a Factor
- "(id + id)" is a Factor
- "((id))" is a Factor

### **4.3 Features This Grammar Supports**

- ✓ Identifiers (variables): id, x, y
- ✓ Parentheses: ( )
- ✓ Addition operator: +
- ✓ Multiplication operator: \*
- ✓ Nested expressions:  $(a + (b * c))$
- ✓ Operator precedence: \* before +

### **4.4 Example Derivations**

#### **Example 1: Derive "id + id"**

Step 1:  $E$  (start)

Step 2:  $E \rightarrow E + T$

Step 3:  $E + T \rightarrow T + T$  (using  $E \rightarrow T$ )

Step 4:  $T + T \rightarrow F + T$  (using  $T \rightarrow F$ )

Step 5:  $F + T \rightarrow id + T$  (using  $F \rightarrow id$ )

Step 6:  $id + T \rightarrow id + F$  (using  $T \rightarrow F$ )

Step 7:  $id + F \rightarrow id + id$  (using  $F \rightarrow id$ )

Complete:  $E \Rightarrow E + T \Rightarrow T + T \Rightarrow F + T \Rightarrow id + T \Rightarrow id + F \Rightarrow id + id$

### Example 2: Derive "id \* id"

Step 1: E (start)

Step 2:  $E \rightarrow T$

Step 3:  $T \rightarrow T * F$

Step 4:  $T * F \rightarrow F * F$  (using  $T \rightarrow F$ )

Step 5:  $F * F \rightarrow id * F$  (using  $F \rightarrow id$ )

Step 6:  $id * F \rightarrow id * id$  (using  $F \rightarrow id$ )

Complete:  $E \Rightarrow T \Rightarrow T * F \Rightarrow F * F \Rightarrow id * F \Rightarrow id * id$

## 5. How the Parser Uses Grammar

### 5.1 Parser's Job

**Input:** Token stream from lexical analyzer

**Process:** Try to build a parse tree using grammar rules

**Output:**

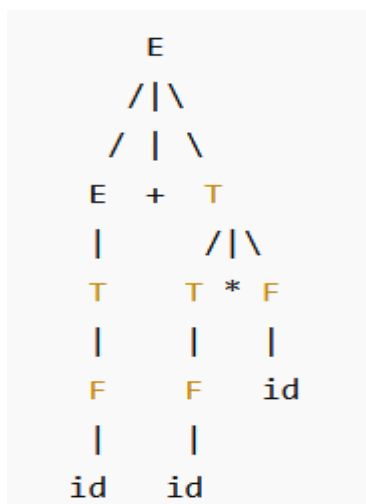
- If successful  $\rightarrow$  Valid syntax (continue compilation)
- If fails  $\rightarrow$  Syntax error (report error and stop)

### 5.2 Parse Tree Construction

**Input Tokens:** id + id \* id (Grammar used here  $E \rightarrow E + T \mid T$

$T \rightarrow T * F \mid F$  and  $F \rightarrow id$

**Parser attempts to build parse tree:**



Reading leaves left to right: id + id \* id ✓

This is a VALID parse tree, so syntax is correct!

**If parser cannot build a valid tree:**

Input: id + \* id

Parser tries to build tree but fails because:

- After "id +", expects a term
- Finds "\*" which cannot start a term
- SYNTAX ERROR reported

Error Message: "Syntax error: unexpected operator '\*' after '+'"

### 5.3 Complete Example

**Valid Input:** id + id \* id

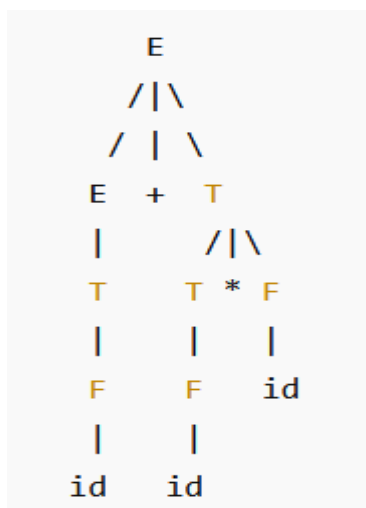
STEP 1: Lexical Analysis

Tokens: ID(x), +, ID(y), \*, ID(z)

STEP 2: Syntax Analysis

Parser builds parse tree using grammar rules

Parse Tree:



Result: VALID SYNTAX ✓

STEP 3: Continue to next phase (semantic analysis)

**Invalid Input:** id + \* id

STEP 1: Lexical Analysis

Tokens: ID(x), +, \*, ID(y)

STEP 2: Syntax Analysis

Parser attempts to build parse tree

After "E + ", expects T (term)

But finds "\*" which cannot start a term

Result: SYNTAX ERROR X

STEP 3: Report error and stop compilation

Error: "Syntax error at line 1: expected expression after '+', found '\*'"

## 6. Types of Parsers

### 6.1 Two Main Categories

#### Category 1: Top-Down Parsers

Strategy: Start from START SYMBOL, work down to input

Process:

- Begin with: S (start symbol)
- Apply productions going downward
- Try to match: input tokens

Direction: Root → Leaves

Use: Leftmost derivations

#### Category 2: Bottom-Up Parsers

Strategy: Start from INPUT, work up to start symbol

Process:

- Begin with: input tokens
- Apply productions in reverse going upward
- Try to reach: S (start symbol)

Direction: Leaves → Root

Use: Rightmost derivations (in reverse)

## 6.2 Comparison

| Aspect      | Top-Down                     | Bottom-Up                    |
|-------------|------------------------------|------------------------------|
| Start       | Start symbol                 | Input tokens                 |
| Direction   | $S \rightarrow \text{input}$ | $\text{Input} \rightarrow S$ |
| Derivation  | Leftmost                     | Rightmost (reverse)          |
| Tree build  | Root $\rightarrow$ leaves    | Leaves $\rightarrow$ root    |
| Power       | Less powerful                | More powerful                |
| Grammar     | Restricted                   | Broader                      |
| Parser type | LL(k)                        | LR(k)                        |

## 7. Derivations

### 7.1 What is a Derivation?

**Definition:** A **derivation** is the step-by-step way of creating a valid program statement by starting from the start symbol and applying grammar rules until only terminals remain.

**Process:**

1. Start with the grammar's start symbol  $S$
2. Repeatedly replace non-terminals using grammar rules
3. Continue until only terminals remain

**Purpose:** Shows how a string can be generated from the grammar.

### 7.2 Leftmost Derivation

**Definition:** In a leftmost derivation, at each step you expand the **leftmost non-terminal first**.

**Rule:** Always replace the leftmost variable in the current sentential form.

**Example Grammar:**

$$E \rightarrow E + E \mid E * E \mid (E) \mid F$$
$$F \rightarrow aF \mid bF \mid 0F \mid 1F \mid \epsilon$$

*Example: Leftmost Derivation of " $a(ab+10)$ "\**

LEFTMOST DERIVATION (always expand leftmost non-terminal):

Step 1: E (start)

Step 2:  $E * E$  (replace leftmost E using  $E \rightarrow E * E$ )

Step 3:  $F * E$  (replace leftmost E using  $E \rightarrow F$ )

Step 4:  $aF * E$  (replace F using  $F \rightarrow aF$ )

Step 5:  $a * E$  (replace F using  $F \rightarrow \epsilon$ )

Step 6:  $a * (E)$  (replace E using  $E \rightarrow (E)$ )

Step 7:  $a * (E + E)$  (replace E using  $E \rightarrow E + E$ )

Step 8:  $a * (F + E)$  (replace leftmost E using  $E \rightarrow F$ )

Step 9:  $a * (aF + E)$  (replace F using  $F \rightarrow aF$ )

Step 10:  $a * (abF + E)$  (replace F using  $F \rightarrow bF$ )

Step 11:  $a * (ab + E)$  (replace F using  $F \rightarrow \epsilon$ )

Step 12:  $a * (ab + F)$  (replace E using  $E \rightarrow F$ )

Step 13:  $a * (ab + 1F)$  (replace F using  $F \rightarrow 1F$ )

Step 14:  $a * (ab + 10F)$  (replace F using  $F \rightarrow 0F$ )

Step 15:  $a * (ab + 10)$  (replace F using  $F \rightarrow \epsilon$ )

Final:  $a * (ab + 10) \checkmark$

Key Point: At each step, we replaced the LEFTMOST non-terminal

### 7.3 Rightmost Derivation

**Definition:** In a rightmost derivation, at each step you expand the **rightmost non-terminal** first.

**Rule:** Always replace the rightmost variable in the current sentential form.

*Example: Rightmost Derivation of " $a(ab+10)^*$ "*

RIGHTMOST DERIVATION (always expand rightmost non-terminal):

Step 1: E (start)

Step 2:  $E * E$  (replace rightmost E using  $E \rightarrow E * E$ )

Step 3:  $E * (E)$  (replace rightmost E using  $E \rightarrow (E)$ )

Step 4:  $E * (E + E)$  (replace E using  $E \rightarrow E+E$ )  
 Step 5:  $E * (E + F)$  (replace rightmost E using  $E \rightarrow F$ )  
 Step 6:  $E * (E + 1F)$  (replace rightmost F using  $F \rightarrow 1F$ )  
 Step 7:  $E * (E + 10F)$  (replace rightmost F using  $F \rightarrow 0F$ )  
 Step 8:  $E * (E + 10)$  (replace rightmost F using  $F \rightarrow \epsilon$ )  
 Step 9:  $E * (F + 10)$  (replace rightmost E using  $E \rightarrow F$ )  
 Step 10:  $E * (aF + 10)$  (replace F using  $F \rightarrow aF$ )  
 Step 11:  $E * (abF + 10)$  (replace F using  $F \rightarrow bF$ )  
 Step 12:  $E * (ab + 10)$  (replace F using  $F \rightarrow \epsilon$ )  
 Step 13:  $F * (ab + 10)$  (replace leftmost E using  $E \rightarrow F$ )  
 Step 14:  $aF * (ab + 10)$  (replace F using  $F \rightarrow aF$ )  
 Step 15:  $a * (ab + 10)$  (replace F using  $F \rightarrow \epsilon$ )  
 Final:  $a * (ab + 10) \checkmark$

Key Point: At each step, we replaced the RIGHTMOST non-terminal

## 7.4 Why Leftmost vs Rightmost Matters

### Usage in Parsers:

TOP-DOWN PARSERS use LEFTMOST derivations

- Start from start symbol
- Expand leftmost non-terminal
- Work down to input

BOTTOM-UP PARSERS use RIGHTMOST derivations in REVERSE

- Start from input
- Reduce rightmost sentential form
- Work up to start symbol

### Important Note:

- Same string can have both leftmost and rightmost derivations
- They produce the same parse tree (if unambiguous)
- The order of expansion differs, not the final result

## 8. Ambiguity in Grammars

### 8.1 Definition of Ambiguity

A grammar is **AMBIGUOUS** if a string has:

- More than one leftmost derivation, OR
- More than one rightmost derivation, OR
- More than one parse tree

**Simple Explanation:** One input can be interpreted in multiple valid ways.

### 8.2 Why Ambiguity is a Problem

#### Problem 1: Multiple Meanings

If "a + b \* c" has two parse trees:

Tree 1: (a + b) \* c = 7 \* c

Tree 2: a + (b \* c) = a + 6

Different trees give different values!

Compiler doesn't know which to use.

#### Problem 2: Unpredictable Behavior

Different compilers might choose different interpretations

Same code could behave differently on different systems

#### Problem 3: Parsing Difficulty

Parser may not know which production to apply

Can lead to inefficient or failing parsers

## 9. Classical Ambiguous Grammar: Dangling Else Problem

### 9.1 The Dangling Else Grammar

**Grammar for Conditional Statements:**

$S \rightarrow \text{if } E \text{ then } S$

    | if E then S else S

    | other

Where:

S = Statement, E = Expression (condition) ,other = any other statement

## 9.4 Why This Happens

**Root Cause:** The grammar does NOT specify:

- Which 'if' the 'else' should bind to
- Whether 'else' matches nearest 'if' or any outer 'if'
- Precedence of 'else' keyword

**Result:** Both interpretations are valid according to the grammar rules.

## 9.5 Real-World Convention

**Most Languages (C, Java, Python, JavaScript) Follow:**

**RULE:** An 'else' is matched with the NEAREST UNMATCHED 'if'

So: if E1 then if E2 then S1 else S2

Means: if E1 then (if E2 then S1 else S2)

The 'else' binds to the inner 'if E2'

## 9.6 Solution: Rewrite Grammar

**Unambiguous Grammar (EXTRA)**

$S \rightarrow \text{matched} \mid \text{unmatched}$

$\text{matched} \rightarrow \text{if } E \text{ then matched else matched}$   
 $\mid \text{other}$

$\text{unmatched} \rightarrow \text{if } E \text{ then } S$   
 $\mid \text{if } E \text{ then matched else unmatched}$

Explanation:

- "matched" = statement with all if's properly matched with else's
- "unmatched" = statement with at least one unmatched if
- This enforces: else always matches nearest if

**Example Parsing with New Grammar:**

Input: if E1 then if E2 then S1 else S2

Derivation (only ONE possible way):

$S \rightarrow \text{unmatched}$

$\rightarrow \text{if } E1 \text{ then } S$

$\rightarrow \text{if } E1 \text{ then matched}$

$\rightarrow \text{if } E1 \text{ then if } E2 \text{ then matched else matched}$

$\rightarrow \text{if } E1 \text{ then if } E2 \text{ then other else other}$

$\rightarrow \text{if } E1 \text{ then if } E2 \text{ then } S1 \text{ else } S2$

Result: else MUST match inner if (only one parse tree possible)

See [Geeks for geeks](#) for Remove ambiguity

## 10. Left Recursion

### 10.1 What is Left Recursion?

**Definition:** Left recursion is a property of a context-free grammar (CFG) where a non-terminal refers to itself as the leftmost symbol in its own production.

**General Form:**

$A \rightarrow A\alpha$

**Simple Explanation:** A production where the non-terminal appears as the first symbol on the right-hand side.

### 10.2 The Problem with Left Recursion

**Why is this a problem?**

Consider:  $A \rightarrow A\alpha$

Leftmost derivation:

Step 1:  $A$

Step 2:  $A \rightarrow A\alpha$  (replace  $A$  with  $A\alpha$ )

Step 3:  $A\alpha \rightarrow A\alpha\alpha$  (replace  $A$  with  $A\alpha$  again)

Step 4:  $A\alpha\alpha \rightarrow A\alpha\alpha\alpha$  (replace  $A$  with  $A\alpha$  again)

...

INFINITE LOOP!

The parser keeps expanding  $A$  with  $A\alpha$  forever .Never reaches a terminal

Top-down parsers FAIL on left-recursive grammars

**Impact:**

- Top-down parsers cannot handle left-recursive grammars
- Causes infinite recursion
- Parser never terminates

### 10.3 Types of Left Recursion

#### TYPE 1: Immediate Left Recursion

Definition: A non-terminal refers to itself directly on the left side

Format:  $A \rightarrow A\alpha \mid \beta$

Example 1:

$E \rightarrow E + T \mid T$

Example 2:

$A \rightarrow Aa \mid b$

Example 3:

$S \rightarrow Sa \mid Sb \mid c$

#### TYPE 2: Indirect Left Recursion

Definition: A non-terminal refers to itself through other non-terminals

Format:

$A \rightarrow B\alpha$

$B \rightarrow A\beta$

Example 1:

$A \rightarrow Ba \mid c$

$B \rightarrow Ab \mid d$

Example 2:

$S \rightarrow Aa$

$A \rightarrow Sb \mid b$

## 11. Removing Immediate Left Recursion

### 11.1 The Transformation Algorithm

**ORIGINAL GRAMMAR (Left Recursive):**

$A \rightarrow A\alpha \mid \beta$

**TRANSFORMED GRAMMAR (No Left Recursion):**

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

### 11.2 How It Works

**Original Grammar:**

$A \rightarrow A\alpha \mid \beta$

Generates strings:  $\beta, \beta\alpha, \beta\alpha\alpha, \beta\alpha\alpha\alpha, \dots$

**Transformed Grammar:**

$A \rightarrow \beta A'$

$A' \rightarrow \alpha A' \mid \epsilon$

Also generates:  $\beta, \beta\alpha, \beta\alpha\alpha, \beta\alpha\alpha\alpha, \dots$

**Key Insight:** Both grammars generate the SAME language, but the transformed version is NOT left-recursive.

### 11.3 Example 1: Remove Left Recursion from $E \rightarrow E + T \mid T$

**STEP 1: Identify the pattern**

Original:

$E \rightarrow E + T \mid T$

Match to pattern  $A \rightarrow A\alpha \mid \beta$ :

-  $A = E$

-  $\alpha = + T$

-  $\beta = T$

**STEP 2: Apply transformation**

$A \rightarrow \beta A'$

$E \rightarrow T E'$

$A' \rightarrow \alpha A' \mid \epsilon$

$E' \rightarrow + T E' \mid \epsilon$

**STEP 3: Final transformed grammar**

$E \rightarrow T E'$

$E' \rightarrow + T E' \mid \epsilon$

**11.4 Example 2: Remove Left Recursion from  $A \rightarrow Aa \mid b$**

**STEP 1: Identify**

$A \rightarrow Aa \mid b$

Pattern:  $A \rightarrow A\alpha \mid \beta$

-  $\alpha = a$

-  $\beta = b$

**STEP 2: Transform**

$A \rightarrow b A'$

$A' \rightarrow a A' \mid \epsilon$

**STEP 3: Verify**

Same! ✓

**11.5 Example 3: Multiple Productions**

**Original Grammar:**

$S \rightarrow Sa \mid Sb \mid c$

**STEP 1: Group by left recursion**

Left-recursive productions:  $Sa, Sb$

Non-left-recursive:  $c$

$$\alpha_1 = a$$

$$\alpha_2 = b$$

$$\beta = c$$

### STEP 2: Apply general transformation

For  $A \rightarrow A\alpha_1 \mid A\alpha_2 \mid \dots \mid A\alpha_n \mid \beta_1 \mid \beta_2 \mid \dots \mid \beta_m$

Transform to:

$$A \rightarrow \beta_1 A' \mid \beta_2 A' \mid \dots \mid \beta_m A'$$

$$A' \rightarrow \alpha_1 A' \mid \alpha_2 A' \mid \dots \mid \alpha_n A' \mid \epsilon$$

### STEP 3: Result

$$S \rightarrow c S'$$

$$S' \rightarrow a S' \mid b S' \mid \epsilon$$

## 12. Removing Indirect Left Recursion

### 12.1 The Problem

**Example of Indirect Left Recursion:**

$$A \rightarrow Ba \mid c$$

$$B \rightarrow Ab \mid d$$

**Why it's left-recursive:**

$$A \rightarrow Ba \rightarrow Aba \text{ (A derives itself after two steps)}$$

### 12.2 Solution Strategy

**STEP 1: Convert indirect to immediate left recursion**

**STEP 2: Remove immediate left recursion**

### 12.3 Complete Example

**Original Grammar (Indirect):**

$$A \rightarrow Ba \mid c$$

$$B \rightarrow Ab \mid d$$

**STEP 1: Eliminate indirect recursion**

Substitute B's productions into A:

$$A \rightarrow Ba \mid c$$

$$\rightarrow (Ab)a \mid (d)a \mid c \quad (\text{substitute } B \rightarrow Ab \mid d)$$

$$\rightarrow Aba \mid da \mid c$$

Reorder:

$$A \rightarrow Aba \mid c \mid da$$

**STEP 2: Now we have immediate left recursion**

$$A \rightarrow Aba \mid c \mid da$$

$$\text{Pattern: } A \rightarrow A\alpha \mid \beta_1 \mid \beta_2$$

$$- \alpha = ba$$

$$- \beta_1 = c$$

$$- \beta_2 = da$$

**STEP 3: Apply transformation**

$$A \rightarrow cA' \mid daA'$$

$$A' \rightarrow baA' \mid \epsilon$$

**STEP 4: Handle B**

$$\text{Original: } B \rightarrow Ab \mid d$$

Substitute new A:

$$B \rightarrow (cA' \mid daA')b \mid d$$

$$B \rightarrow cA'b \mid daA'b \mid d$$

**FINAL TRANSFORMED GRAMMAR:**

$$A \rightarrow cA' \mid daA'$$

$$A' \rightarrow baA' \mid \epsilon$$

$$B \rightarrow cA'b \mid daA'b \mid d$$

### 13. Left factoring

Left factoring is a grammar transformation technique used to remove common prefixes from productions so that a parser can make a single, deterministic choice



#### Problem-01:

Do left factoring in the following grammar-

$$S \rightarrow iEtS / iEtSeS / a$$
$$E \rightarrow b$$

#### Solution-

The left factored grammar is-

$$S \rightarrow iEtSS' / a$$
$$S' \rightarrow eS / \epsilon$$
$$E \rightarrow b$$

#### Problem-02:

Do left factoring in the following grammar-

$$A \rightarrow aAB / aBc / aAc$$

#### Solution-

##### Step-01:

$$A \rightarrow aA'$$
$$A' \rightarrow AB / Bc / Ac$$

Again, this is a grammar with common prefixes.

**Step-02:**

$$A \rightarrow aA'$$

$$A' \rightarrow AD / Bc$$

$$D \rightarrow B / c$$

This is a left factored grammar.

**Problem-03:**

Do left factoring in the following grammar-

$$S \rightarrow bSSaS / bSSaSb / bSb / a$$

**Solution-**

**Step-01:**

$$S \rightarrow bSS' / a$$

$$S' \rightarrow SaaS / SaSb / b$$

Again, this is a grammar with common prefixes.

**Step-02:**

$$S \rightarrow bSS' / a$$

$$S' \rightarrow SaA / b$$

$$A \rightarrow aS / Sb$$

This is a left factored grammar.

**Problem-04:**

Do left factoring in the following grammar-

$$S \rightarrow aSSbS / aSaSb / abb / b$$

**Solution-**

**Step-01:**

$$S \rightarrow aS' / b$$

$$S' \rightarrow SSbS / SaSb / bb$$

Again, this is a grammar with common prefixes.

**Step-02:**

$$S \rightarrow aS' / b$$

$$S' \rightarrow SA / bb$$

$$A \rightarrow SbS / aSb$$

This is a left factored grammar.

**Problem-05:**

Do left factoring in the following grammar-

$$S \rightarrow a / ab / abc / abcd$$

**Solution-**

**Step-01:**

$$S \rightarrow aS'$$

$$S' \rightarrow b / bc / bcd / \epsilon$$

Again, this is a grammar with common prefixes.

**Step-02:**

$$S \rightarrow aS'$$

$$S' \rightarrow bA / \epsilon$$

$$A \rightarrow c / cd / \epsilon$$

Again, this is a grammar with common prefixes.

**Step-03:**

$$S \rightarrow aS'$$
$$S' \rightarrow bA / \epsilon$$
$$A \rightarrow cB / \epsilon$$
$$B \rightarrow d / \epsilon$$

This is a left factored grammar.

#### **Problem-06:**

Do left factoring in the following grammar-

$$S \rightarrow aAd / aB$$
$$A \rightarrow a / ab$$
$$B \rightarrow ccd / ddc$$

#### **Solution-**

The left factored grammar is-

$$S \rightarrow aS'$$
$$S' \rightarrow Ad / B$$
$$A \rightarrow aA'$$
$$A' \rightarrow b / \epsilon$$
$$B \rightarrow ccd / ddc$$

### **Annotated Parse Tree**

The parse tree containing the values of attributes at each node for given input string is called annotated or decorated parse tree.

#### **Features of Annotated Parse Trees**

- High level specification
- Hides implementation details
- Explicit order of evaluation is not specified

**Example:** Consider the following grammar

$$\begin{aligned} S &\rightarrow E \\ E &\rightarrow E + T \end{aligned}$$

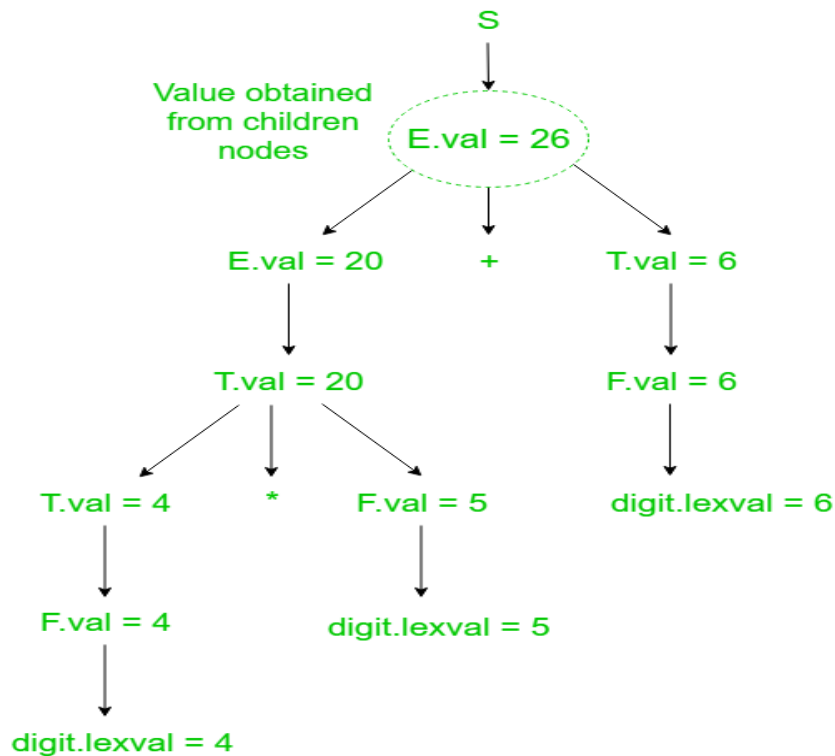
```

E --> T
T --> T * F
T --> F
F --> digit

```

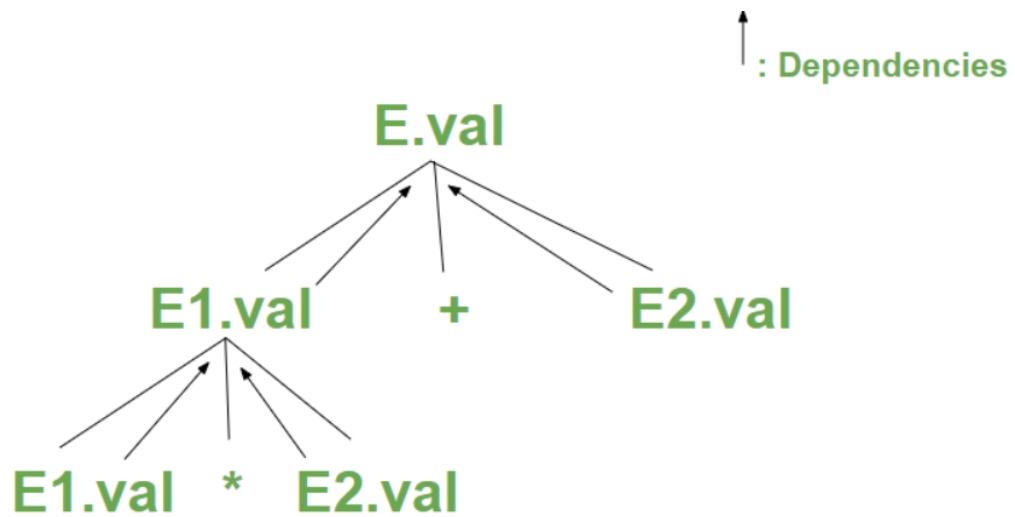
The SDD for the above grammar can be written as follow

Let us assume an input string **4 \* 5 + 6** for computing synthesized attributes. The annotated parse tree for the input string is



**Annotated Parse Tree**

## Dependency Graph



Dependency Graph for the above example

## Information Stored in a Symbol Table

Based on your table, a symbol table typically contains:

| Field               | Description                                          |
|---------------------|------------------------------------------------------|
| Name                | Identifier name (e.g., x, a)                         |
| Type                | Data type (int, char, float, etc.)                   |
| Size                | Memory size required (e.g., int = 4 bytes)           |
| Dimension           | Number of dimensions (0 = variable, 1 = array, etc.) |
| Line of Declaration | Source code line where identifier is declared        |
| Line of Usage       | Lines where identifier is used                       |
| Address             | Memory location assigned to the identifier           |